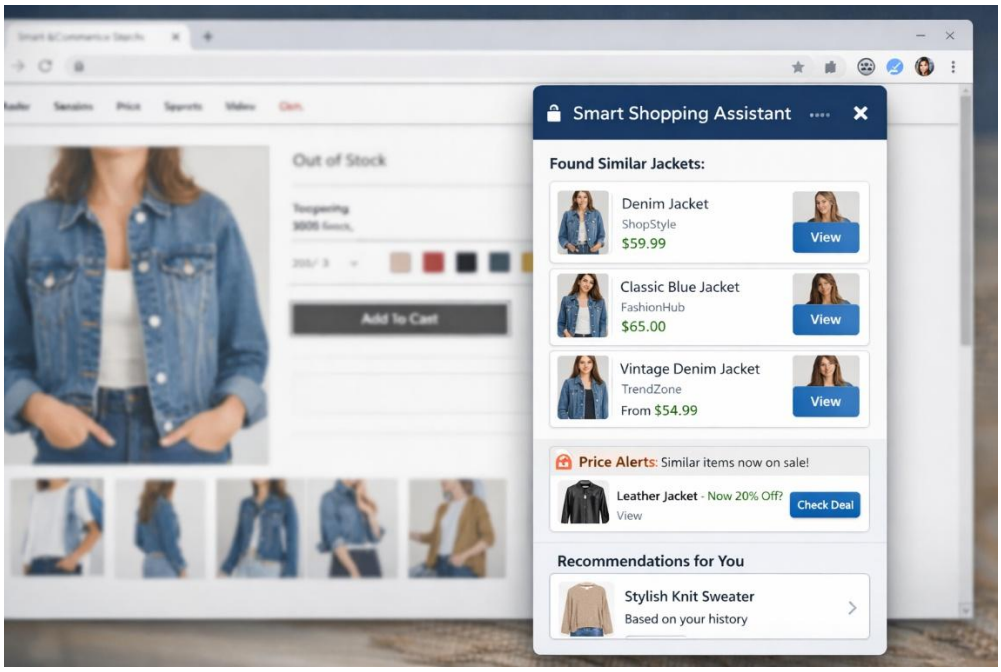
Software Product - Smart Shopping Extension

The goal of this project is to develop a smart Chrome extension that assists users in making better online shopping decisions.

Mor Greenberg 212465678

Stav Glam 322331984



Introduction

Our project aims to improve the online fashion shopping experience by automatically searching for out-of-stock fashion items across multiple relevant websites.
In line with the workshop requirements, the project is a software-based product that delivers clear value to users by saving time, improving product availability discovery, and potentially reducing costs. By simplifying the search process and exposing users to a wider range of brands and online stores, the product improves the overall shopping experience and demonstrates practical practical value and business potential.

Specification

The Need

Consumers who shop for fashion online often encounter items that are out of stock or unavailable on a specific website. Finding the same or similar items on other platforms requires repetitive, time-consuming manual searches across multiple sites.

The Solution

The proposed solution is a smart Chrome extension that enables users to quickly locate fashion items they previously searched for, viewed, or liked on one online store across other relevant websites.
By automating cross-site searches and integrating smart features, the system provides a more efficient, convenient, and informed shopping experience, reducing both time and effort for users.

User Story

As an online fashion shopper,
I want to easily find an item I liked but is out of stock on another website,
so that I can save time, discover alternative stores or prices, and make better purchasing decisions without manually searching multiple sites.

Features

Short-term history storage on the client side, enabling the system to suggest fashioitems found via cross-site search after some time and allowing offline search in addition to live search.

Login functionality (from a software product perspective, including user management and actions such as email notifications).

Autonomous price comparison and promotion suggestions.

Mobile compatibility (optional).

AI fashion advisor – a recommendation system based on leading trends, clothing style, and matching with additional items (optional).

Expand to other shopping fields (optional)

Flow

User browses an online fashion store.

1. User views or selects a specific item.
2. The Chrome extension is activated.
3. The system performs an automated cross-site search for the item.
4. Results are displayed, including availability, prices, and alternative options.
5. User selects a preferred option or saves the item for later.

Scenarios

Scenario 1 – Out-of-Stock Item

A user finds a jacket they like, but it is out of stock. By activating the extension, the system automatically finds the same jacket on other websites where it is available.

Scenario 2 – Price Comparison

A user activates the extension to compare prices of an item across multiple stores and chooses the most cost-effective option.

Scenario 3 – Offline Browsing

A user opens the extension without an internet connection and reviews previously searched items from local history.

Scenario 4 – Personalized Recommendation (Optional)

Based on previous searches, the system suggests similar items or complementary fashion pieces aligned with the user's style.

Design

The system is designed as a client–server architecture, composed of three main components. This modular structure allows scalability and future extension into additional services if required.

Client Side + UI (TypeScript)

The client side is implemented as a Chrome extension written in TypeScript.
It is responsible for user interaction and lightweight logic, including:

- Detecting viewed or selected fashion items on supported websites
- Triggering cross-site search requests
- Displaying search results, price comparisons, and recommendations
- Managing short-term search history stored locally
- Supporting offline browsing of previously searched items

The user interface appears inside the browser and allows users to shop easily without interrupting their browsing experience.

Server Side (Java)

The server is implemented in Java and handles the core business logic of the system, including:

- Processing cross-site search requests
- Aggregating and normalizing results from multiple fashion websites
- Performing price comparison and promotion analysis
- Managing user accounts and authentication (if enabled)
- Supporting recommendation logic based on user behavior and trends

The server exposes a RESTful API that is consumed by the Chrome extension.

Database

The system uses two separate databases, as shown in the system diagram.

Users Data Database

This database stores user-related information and is accessed by the server mainly when users are logged in.

Stored data includes:

- User account details
- User preferences
- Search history

Example tables:

- Users (user_id, email, password_hash)
- UserPreferences (user_id, preferences)
- SearchHistory (user_id, item_details, timestamp)

Fashion Items Data Database

This database stores data related to fashion items collected from different shopping websites.
It is used by the search server to perform cross-site searches and comparisons.

Stored data includes:

- Item details (name, category, brand)
- Price information
- Availability status
- Source website

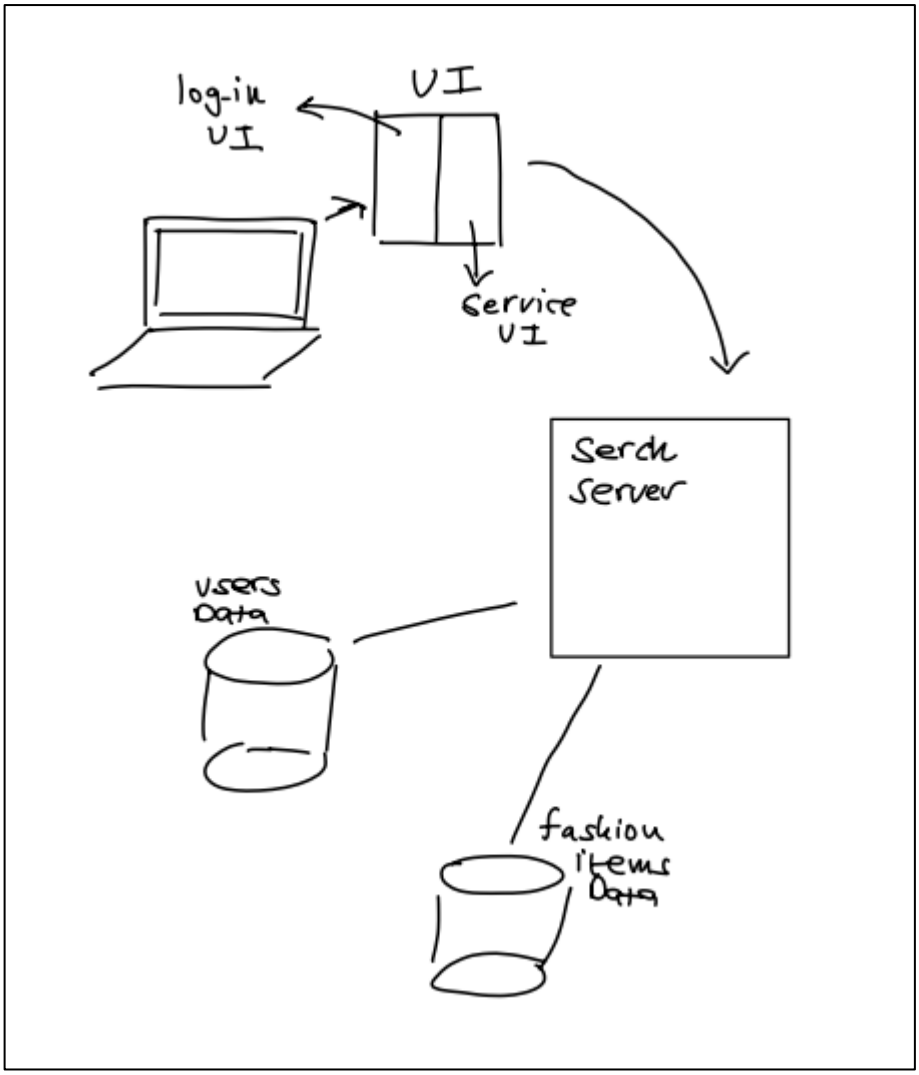
Example tables:

- **FashionItems** (item_id, name, brand, category)
- **ItemPrices** (item_id, website, price, availability, timestamp)

External Tools & Libraries

The system relies on the following tools and technologies, according to the system architecture:

- **TypeScript & Chrome Extension APIs** – for implementing the client-side extension and UI
- **Java (Spring Boot)** – for implementing the search server and business logic
- **HTTP / REST APIs** – for communication between the Chrome extension and the server
- **Database Management Systems** – used for storing user data and fashion item data (relational or NoSQL, depending on scalability needs)


log-in
UI
UI
Service
UI
Serch
Server
Users
Data
fashion
items
Data